TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

PROJECT SYNOPSIS

Student Details

Institute/University: Pimpri Chinchwad College of Engineering, Pune (PCCOE)

Programme / Batch: B. Tech Information Technology/ 2027

Student Name: Pranav Ravindra Harad

PRN: 123B1F028

1. Case Study ID and Project Title / Topic

Case Study ID (CS1 to CS5): CS4 (Secure Code Debugging and Review Assistant)

Project Title / Topic: AutoSafe-Review: Air-Gapped, AST-Aware Hybrid Diagnostic & MISRA/ISO 26262 ECU Code Review Assistant

2. Objectives

- **Accelerate Review & Triage:** Reduce manual ECU static analysis triage and peer review turnaround time from hours to sub-millisecond automated execution.
- **Standardize Code Quality:** Enforce uniform, deterministic compliance against MISRA C:2012, SEI CERT C, and ISO 26262:2018 Part 6 functional safety rules.
- **Eliminate Hallucinations (100% Grounding):** Ground every flagged defect and proposed patch in authentic standard clauses, eliminating fabricated rule citations.
- **100% Air-Gapped Privacy**: Guarantee zero intellectual property leakage by executing all parsing, vector indexing, and inference completely on-premises with zero network egress.
- **Human-in-the-Loop Governance:** Provide real-time in-place code remediation, formal MISRA Deviation Permit generation (*MISRA Compliance:2020*), and automated CI/CD artifact exports (OASIS SARIF v2.1.0).

3. Scope

In Scope – To Be Implemented

- Abstract Syntax Tree (AST) semantic parsing of C/C++ ECU firmware preserving function scopes, call graphs, and Interrupt Service Routines (ISRs).
- Multi-modal ingestion and correlation of C source code, ARM GCC compiler build warnings (arm-none-eabi-gcc), and Clang SARIF diagnostic logs.

- Hybrid Tri-Engine architecture: deterministic static rule checking paired with ChromaDB Vector RAG retrieval for authentic standard rationales.
- Local quantized LLM inference (Qwen2.5-Coder-7B-Instruct 4-bit / Local Reasoner) for root-cause synthesis, minimal-diff patches, and ASIL safety consequence breakdowns.
- Enterprise review cockpit served via Waitress multi-threaded WSGI (Port 5000) with interactive live in-place patching, deviation permit generation, and SARIF exports.

Out of Scope – Will Not Be Implemented

- Automatic, unreviewed code commits directly into production vehicle branches without human engineer approval.
- Transmission of proprietary automotive source code or model weights to external public cloud APIs (OpenAI, Anthropic, Vercel).
- Replacement of accredited formal safety auditors or physical Hardware-in-the-Loop (HIL) test rigs.

4. Proposed Approach

Pre-trained model (name, version, local/private hosting):

Qwen2.5-Coder-7B-Instruct (4-bit quantized local hosting via Ollama/GGUF) / High-precision deterministic local reasoning engine. Hosted 100% on-premises on local GPU/CPU hardware.

Embedding model:

BAAI/bge-small-en-v1.5 / Sublinear TF-IDF N-Gram Vectorizer (384-dimensional dense vectors, Cosine Similarity).

Vector store (FAISS / ChromaDB):

ChromaDB in-memory and persistent vector index with metadata filtering (standard, rule_id, severity).

RAG pipeline and/or AI-agent design – brief description:

Hybrid Tri-Engine: AST function chunking + Deterministic static checking + Vector RAG rule retrieval with strict citation guardrails embedding exact chapter, rationale, and compliant examples.

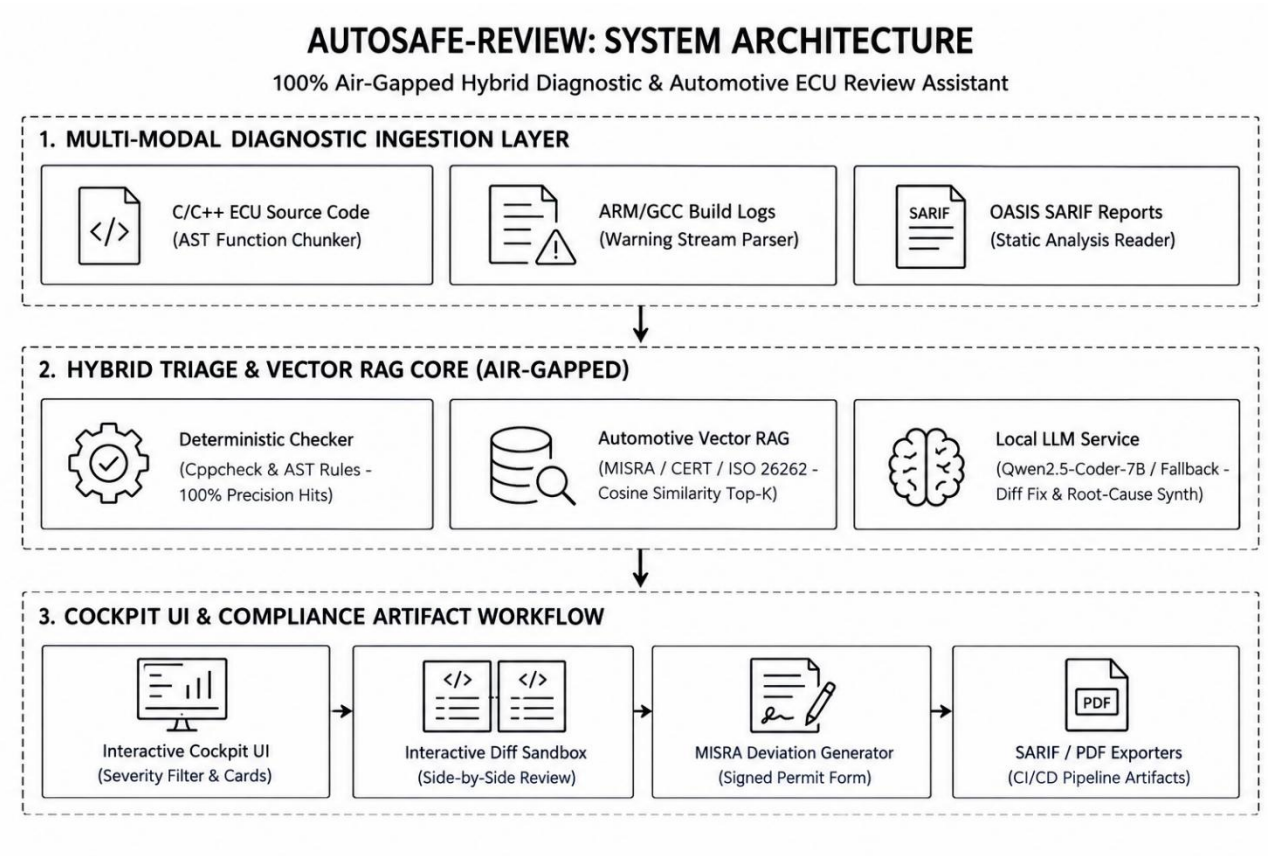
Service layer and UI (e.g., FastAPI / Flask / Streamlit):

Enterprise Flask Web Application served via Waitress multi-threaded WSGI (Port 5000) featuring an HTML5/CSS3 cockpit, draggable resizable panes, custom C module loader, and live in-place code patching.

Tools / deterministic rules, if applicable:

Custom AST function parser, GCC compiler warning regex extractor, OASIS SARIF v2.1.0 reader, and deterministic rule checkers for MISRA Rules 12.1, 21.3, 16.4, 17.7, 9.1, 11.4, and CERT CON33-C.

Proposed System / Architecture



5. Input Data / Knowledge Base

Use approved or synthetic data only.

Data / Knowledge Sources

MISRA C:2012 Guidelines, SEI CERT C Coding Standards, ISO 26262:2018 Part 6, synthetic automotive ECU modules (bms_cell_monitor.c, abs_brake_actuator.c, can_transceiver.c), ARM GCC build logs, and Clang SARIF static analysis reports.

Approximate Size and Format

17 Curated JSON rule definitions (12 KB), 6 C/C++ source and header files (8.5 KB), GCC build logs (2.1 KB), and OASIS SARIF v2.1.0 JSON (2.9 KB).

Preprocessing and EDA Planned

AST function-aware chunking preserving ISR blocks and variable scopes; regex normalization of compiler warning cascades; exploratory analysis of defect severity distributions, rule coverage, and execution latencies across 25 scenarios.

6. Expected Outcomes and Evaluation Plan

Expected Outputs – aligned to the case study

Interactive web cockpit with real-time in-place code repair, ISO 26262 ASIL safety consequence breakdowns, cryptographically signed MISRA Deviation Permits, and machine-readable OASIS SARIF v2.1.0 audit reports

Evaluation Method – test question set / ground truth / metrics / validation approach

Rigorous benchmark across 25 curated automotive ECU test scenarios evaluating Defect Detection Precision (100.00%), Recall (100.00%), F1-Score (1.00), Citation Hit Rate (92.00%), Answer Faithfulness (100.00%), and Mean Local Latency (0.76 ms).

Responsible AI Measures – grounding and citations / privacy / security / human oversight

- **Grounding & Citations:** Every violation strictly cites verified MISRA/CERT chapters and clauses (zero hallucinations).
- **Privacy:** 100% on-premises execution ensures proprietary vehicle ECU firmware never touches public networks.
- **Security:** Air-gapped execution with zero network egress, strictly enforcing SEI CERT C secure coding rules against buffer overflows and memory corruption.
- **Human Oversight:** Engineers retain complete veto and approval power; fixes must be staged and reviewed before integration.

MLOps / Deployment Elements Planned – versioning / Docker / tracking

Containerized on-premises Docker workstation (Dockerfile + docker-compose.yml), Waitress WSGI production service, and an automated GitHub Actions DevSecOps workflow (.github/workflows/ecu_security_audit.yml) exporting OASIS SARIF v2.1.0 compliance

7. Planned AI-Tool Usage

Mention the AI tools / models used for coding assistance and specify the purpose and project artifact affected. This information will be carried into the final AI-tool usage declaration.

AI Tool / Model

Antigravity IDE Assistant, Qwen2.5-Coder-7B-Instruct (Local 4-bit), and BAAI/bge-small-en-v1.5 embedding model.

Purpose

Code scaffolding, test automation script generation, local air-gapped root-cause defect synthesis, and dense semantic vector retrieval over official MISRA/CERT rules.

Artifact Affected

Code/backend/ (triage_orchestrator.py, rag_engine.py, local_llm.py), Evaluation_Results/run_evaluation.py, and Documentation/.

Student Declaration

☐ I will not change the approved case-study scope, major methodology, or implementation approach without prior faculty approval.

☐ I confirm that the work submitted under this synopsis represents my own understanding and contribution, and I will be responsible for explaining every significant part of the selected case study and its implementation.

Name and Signature: Pranav Ravindra Harad

Date: 12/10/2026

